

Università degli Studi di Catania

Corso di Laurea in Informatica

Progetto di **Sistemi Cloud**

Mensa Universitaria

Applicazione a microservizi:
deployment su cluster Kubernetes locale e migrazione su AWS

Candidato
Vito Marino

Anno Accademico 2025/2026

Indice

1	Introduzione	5
1.1	Obiettivi del progetto	5
1.2	Requisiti	5
1.3	Struttura del documento	5
2	Architettura dell'applicazione	6
2.1	Il dominio	6
2.2	Suddivisione in microservizi	6
2.3	Persistenza dei dati	6
2.4	Flusso di un ordine	7
2.5	Configurazione esterna al codice	7
2.6	Containerizzazione	8
3	Fase 1: deployment su cluster Kubernetes locale	9
3.1	Obiettivo e struttura	9
3.2	Provisioning con Terraform e Multipass	9
3.3	Il collegamento tra Terraform e Ansible	9
3.4	Configurazione del cluster con Ansible	10
3.5	Manifest Kubernetes	10
3.6	Distribuzione delle immagini senza registro	11
3.7	Verifica del deployment	11
4	Fase 2: migrazione su Amazon Web Services	12
4.1	Architettura di destinazione	12
4.2	Scelta tra EKS e cluster autogestito	12
4.3	Rete e sicurezza	12
4.4	Servizi gestiti	14
4.5	Gestione delle credenziali	15
4.6	Gestione dello stato di Terraform	15
4.7	Registro delle immagini	16
4.8	Bilanciamento del carico	16
4.9	Adattamenti al playbook Ansible	16
4.10	Pipeline di integrazione e distribuzione continua	16
4.10.1	Autenticazione senza credenziali statiche	16
4.10.2	Esecuzione del deployment	17
4.11	Automazione dell'avvio	17

5	Confronto, problemi affrontati e conclusioni	18
5.1	Corrispondenza tra le due fasi	18
5.2	Problemi affrontati	18
5.2.1	Concorrenza nella creazione dello schema	18
5.2.2	Assenza di ordinamento all'avvio in Kubernetes	19
5.2.3	Schema condiviso tra servizi	19
5.2.4	Vincoli dell'ambiente di sviluppo	19
5.2.5	Non determinismo nella selezione dell'immagine	20
5.2.6	Identificatori immutabili nel token OIDC	20
5.2.7	Visibilità delle immagini su S3	20
5.3	Considerazioni sui costi	21
5.4	Sviluppi possibili	21
5.5	Conclusioni	21
A	Listati principali	22
A.1	order-service	22
A.2	menu-service	25
A.3	kitchen-service	28
A.4	Infrastruttura locale	29
A.5	Playbook Ansible	31
A.6	Infrastruttura AWS	33
A.7	Pipeline CI/CD	44
A.8	Script di deployment	46

Capitolo 1

Introduzione

1.1 Obiettivi del progetto

Il progetto realizza un'applicazione web per la gestione degli ordini di una mensa universitaria e ne studia il deployment in due contesti differenti: un cluster Kubernetes installato su macchine virtuali locali e un'infrastruttura cloud basata su AWS.

L'obiettivo non è soltanto far funzionare l'applicazione, ma verificare una proprietà precisa: che lo stesso identico codice applicativo possa essere eseguito in ambienti profondamente diversi senza modifiche, a patto che la configurazione sia esterna al codice. Questa proprietà viene dimostrata nella seconda fase, dove il passaggio dai container locali ai servizi gestiti di AWS avviene cambiando esclusivamente variabili d'ambiente.

1.2 Requisiti

La traccia richiedeva:

- un'applicazione 3-tier basata su microservizi, con database relazionale e NoSQL, ed eventuale comunicazione asincrona tramite code di messaggi;
- containerizzazione con Docker e orchestrazione con Kubernetes;
- un primo deployment su un cluster Kubernetes reale in locale, realizzato con Multipass;
- la gestione dell'infrastruttura tramite Infrastructure as Code, con Terraform e Ansible;
- la migrazione dell'architettura su AWS utilizzando servizi gestiti;
- una pipeline di integrazione e distribuzione continua con GitHub Actions (nella versione Cloud).

1.3 Struttura del documento

Il documento è organizzato in due parti. Il Capitolo 2 descrive l'applicazione, che è comune a entrambe le fasi. Il Capitolo 3 tratta il deployment locale, il Capitolo 4 la migrazione su AWS. Il Capitolo 5 mette a confronto le due soluzioni e raccoglie i problemi incontrati durante lo sviluppo, che rappresentano una parte significativa del lavoro svolto. Le appendici riportano i listati principali.

Il codice è distribuito in due repository:

- <https://github.com/vitomarino02-del/Cloud-Mensa> (versione locale);
- <https://github.com/vitomarino02-del/Cloud-Mensa-AWS-Version> (versione AWS).

Capitolo 2

Architettura dell'applicazione

2.1 Il dominio

L'applicazione modella il funzionamento di una mensa universitaria. Uno studente consulta il menù del giorno, seleziona i piatti e invia un ordine; il personale di cucina visualizza gli ordini attivi su un tabellone e ne fa avanzare lo stato fino alla consegna. Gli stati previsti sono quattro: *ricevuto*, *in preparazione*, *pronto*, *ritirato*.

Il dominio, pur semplice, presenta tutte le caratteristiche utili a giustificare un'architettura a microservizi: entità con cicli di vita diversi (il catalogo dei piatti cambia raramente, gli ordini continuamente), esigenze di lettura molto frequenti (il display di cucina) e la necessità di notificare eventi ad altri componenti (l'arrivo di un nuovo ordine).

2.2 Suddivisione in microservizi

L'applicazione è suddivisa in tre servizi backend e un frontend.

menu-service. Gestisce il catalogo dei piatti e le relative immagini. Espone le operazioni di lettura del menù e di caricamento delle foto. La responsabilità è circoscritta: non conosce l'esistenza degli ordini.

order-service. È il componente centrale. Riceve gli ordini, li valida, ne calcola il totale e ne gestisce l'avanzamento di stato. È l'unico servizio che scrive su tutti e tre i sistemi di persistenza.

kitchen-service. Non espone alcuna API: è un consumatore di eventi. Resta in ascolto sulla coda dei messaggi e reagisce agli eventi di creazione e di completamento degli ordini. Nella versione attuale registra le notifiche nei log; rappresenta il punto di innesto naturale per un sistema di notifiche reale.

frontend. Single Page Application in React, compilata e servita da Nginx. Nginx svolge anche la funzione di gateway applicativo, instradando le richieste `/api/menu` e `/api/orders` verso i rispettivi servizi. Questa scelta fa sì che l'unico componente esposto verso l'esterno sia il frontend: i microservizi e i sistemi di persistenza non sono mai raggiungibili direttamente.

2.3 Persistenza dei dati

Il progetto utilizza tre sistemi di persistenza distinti, ciascuno con un ruolo preciso.

PostgreSQL è la sorgente di verità. Contiene il catalogo dei piatti, gli ordini e le relative righe, con vincoli di integrità referenziale e transazioni. La struttura prevede tre tabelle: `dishes`, `orders` e `order_items`.

Un dettaglio progettuale rilevante riguarda `order_items`: il nome e il prezzo unitario del piatto vengono *copiati* al momento dell'ordine invece di essere referenziati. In questo modo una successiva variazione di prezzo nel catalogo non altera lo storico degli ordini già effettuati.

Redis ospita il tabellone della cucina. Il display richiede gli ordini attivi con frequenza elevata; interrogare PostgreSQL a ogni aggiornamento significherebbe eseguire ripetutamente una query con join su tre tabelle per ottenere un dato che cambia di rado. Redis mantiene invece una struttura di tipo hash (`kitchen:board`) in cui la chiave è l'identificativo dell'ordine e il valore è la sua rappresentazione JSON già serializzata: la lettura dell'intero tabellone è una singola operazione in memoria e riguarda soltanto gli ordini ancora da evadere.

È importante sottolineare che Redis non contiene dati critici: tutto ciò che vi è memorizzato è ricostruibile da PostgreSQL. Coerentemente, il codice tratta l'indisponibilità di Redis come una condizione di degrado e non di errore.

RabbitMQ realizza la comunicazione asincrona. Quando un ordine viene creato, order-service pubblica un evento su un exchange di tipo *fanout* chiamato `order.events`. La scelta del fanout è deliberata: il produttore non conosce i consumatori, e un eventuale nuovo servizio (statistiche, notifiche push, integrazione contabile) può iscriversi all'exchange senza che il codice di order-service venga modificato.

2.4 Flusso di un ordine

Il percorso completo di un ordine attraversa tutti i componenti descritti:

1. il browser invia `POST /api/orders` a Nginx;
2. Nginx, in base al percorso, inoltra la richiesta a order-service;
3. order-service verifica l'esistenza dei piatti e calcola il totale *lato server*, senza fidarsi dei valori inviati dal client;
4. l'ordine viene salvato su PostgreSQL con un commit transazionale;
5. l'ordine viene inserito nel tabellone Redis;
6. viene pubblicato l'evento `order.created` su RabbitMQ;
7. kitchen-service, in ascolto, riceve l'evento e notifica la cucina;
8. il display di cucina, interrogando `GET /api/orders?active=1`, legge il tabellone da Redis e mostra il nuovo ordine.

L'ordine delle operazioni 4, 5 e 6 non è casuale: la scrittura sul database avviene per prima perché è l'unica che non può essere persa. Le due successive interessano sistemi ausiliari e sono gestite in modo che un loro fallimento non comprometta l'ordine già registrato.

2.5 Configurazione esterna al codice

Ogni servizio legge la propria configurazione da variabili d'ambiente:

```
1 STORAGE_BACKEND = os.environ.get("STORAGE_BACKEND", "local")
2 IMAGE_DIR = os.environ.get("IMAGE_DIR", "/data/images")
3 app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
4     "DATABASE_URL", "postgresql://mensa:mena@localhost:5432/mensa")
```

Listing 2.1: Lettura della configurazione in menu-service

Questa impostazione, apparentemente marginale, è ciò che rende possibile l'intera seconda fase del progetto: gli stessi container che in locale si collegano ai pod di PostgreSQL, Redis e RabbitMQ, su AWS si collegano a RDS, ElastiCache e Amazon MQ senza alcuna modifica al codice.

Lo stesso principio governa la gestione delle immagini dei piatti. Il menu-service espone due funzioni, `save_image` e `image_url`, il cui comportamento dipende dalla variabile `STORAGE_BACKEND`: con il valore `local` i file vengono scritti su un volume, con il valore `s3` vengono caricati su un bucket Amazon S3 tramite la libreria `boto3`.

2.6 Containerizzazione

Ogni servizio è distribuito come immagine Docker autonoma, contenente il proprio codice, le proprie dipendenze e le istruzioni di avvio. La struttura di ciascuna directory di servizio è volutamente identica: il file applicativo, `requirements.txt` con le versioni bloccate e il `Dockerfile`.

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 EXPOSE 5001
7 CMD ["gunicorn", "-b", "0.0.0.0:5001", "-w", "2", "app:app"]
```

Listing 2.2: Dockerfile di menu-service

L'ordine delle istruzioni non è indifferente: copiando `requirements.txt` e installando le dipendenze *prima* di copiare il codice, il livello contenente le librerie viene riutilizzato dalla cache quando cambia soltanto il codice applicativo, riducendo sensibilmente i tempi di ricostruzione.

Il frontend adotta invece una costruzione *multi-stage*: un primo stadio basato su Node compila la SPA, un secondo stadio basato su Nginx ne serve i file statici. L'immagine finale non contiene Node né le dipendenze di sviluppo.

Per l'esecuzione in fase di sviluppo è disponibile un file `docker-compose.yml` che avvia i sette container necessari, definendo la rete interna, i volumi persistenti e le condizioni di avvio: i servizi applicativi attendono che i sistemi di persistenza rispondano ai rispettivi *healthcheck*.

Capitolo 3

Fase 1: deployment su cluster Kubernetes locale

3.1 Obiettivo e struttura

La prima fase prevede il deployment dell'applicazione su un cluster Kubernetes reale, installato su macchine virtuali locali. L'infrastruttura è descritta interamente come codice: Terraform crea le macchine virtuali, Ansible le configura trasformandole in un cluster, i manifest Kubernetes descrivono l'applicazione.

3.2 Provisioning con Terraform e Multipass

Terraform è utilizzato per creare tre macchine virtuali Ubuntu tramite Multipass: una destinata al control plane e due ai nodi worker. Le risorse sono descritte in modo dichiarativo:

```
1 resource "multipass_instance" "worker" {
2   count          = var.worker_count
3   name           = "mensa-worker-${count.index + 1}"
4   image          = "22.04"
5   cpus           = var.cpus
6   memory         = var.memory
7   disk           = var.disk
8   cloudinit_file = local_file.cloudinit.filename
9 }
```

Listing 3.1: Definizione dei nodi worker

Il dimensionamento (due CPU e due gigabyte di memoria per nodo) non è arbitrario: sono i requisiti minimi verificati da kubeadm durante i controlli preliminari.

Al primo avvio ciascuna macchina esegue uno script *cloud-init* che crea l'utente di servizio e vi installa la chiave SSH pubblica dell'operatore. Questo passaggio è ciò che consente ad Ansible di collegarsi successivamente senza password.

3.3 Il collegamento tra Terraform e Ansible

Gli indirizzi IP assegnati alle macchine virtuali non sono noti prima della loro creazione. Terraform li legge al termine del provisioning e genera automaticamente il file di inventario di Ansible:

```
1 resource "local_file" "ansible_inventory" {
2   filename = "${path.module}/ansible/inventory.ini"
```

```

3  content = templatefile("${path.module}/inventory.tpl", {
4      cp_ip      = multipass_instance.control_plane.ipv4
5      worker_ips = [for w in multipass_instance.worker : w.ipv4]
6      ssh_key_path = pathexpand(var.ssh_private_key_path)
7  })
8  }

```

Listing 3.2: Generazione dell'inventario

Il risultato è una catena completamente automatica: Terraform crea le macchine, cloud-init vi installa la chiave, Terraform scrive l'inventario, Ansible lo legge e si collega. Nessun indirizzo viene trascritto manualmente.

3.4 Configurazione del cluster con Ansible

Il playbook `site.yml` è organizzato in tre *play* corrispondenti alle tre fasi logiche dell'installazione.

Il primo play riguarda tutti i nodi e ne prepara il sistema operativo: disattiva la memoria di swap (rifiutata da kubelet), carica i moduli del kernel `overlay` e `br_netfilter`, imposta i parametri di rete necessari all'instradamento tra pod, installa il runtime dei container e i pacchetti `kubelet`, `kubeadm` e `kubect1`.

È opportuno soffermarsi sulla scelta del runtime. Sui nodi non viene installato Docker ma **containerd**. Kubernetes non dialoga direttamente con Docker: utilizza l'interfaccia standard CRI, e containerd la implementa nativamente. Docker resta necessario soltanto sulla macchina di sviluppo, dove le immagini vengono costruite.

Il secondo play inizializza il control plane con `kubeadm init`, installa la rete dei pod (Flannel), genera il comando di join e scarica il file `kubeconfig`. Il terzo play esegue il join sui nodi worker.

```

1  - name: kubeadm init
2    command: >
3      kubeadm init
4      --pod-network-cidr=10.244.0.0/16
5      --apiserver-advertise-address={{ ansible_host }}
6  args:
7    creates: /etc/kubernetes/admin.conf

```

Listing 3.3: Inizializzazione del control plane

L'attributo `creates` merita attenzione: `kubeadm init` non è un comando idempotente, e rieseguirlo su un cluster già inizializzato produrrebbe un errore. Indicando un file che il comando genera, Ansible verifica la presenza di tale file e, se esiste, salta l'operazione. L'intero playbook può quindi essere rieseguito senza effetti collaterali, che è la proprietà fondamentale degli strumenti di configuration management.

3.5 Manifest Kubernetes

I manifest descrivono lo stato desiderato dell'applicazione. Per ciascun servizio sono definiti un **Deployment**, che mantiene attivo un numero prestabilito di repliche, e un **Service**, che fornisce un nome DNS stabile e distribuisce il traffico tra le repliche disponibili.

La distinzione tra i due oggetti è essenziale: i pod sono entità effimere, con indirizzi IP che cambiano a ogni ricreazione; il Service costituisce il punto di accesso stabile. Se un pod termina, il Deployment ne crea immediatamente un altro e il Service smette di indirizzargli traffico: è il meccanismo di *self-healing* che distingue Kubernetes da un semplice orchestratore di container.

Il servizio `order-service` è configurato con due repliche, a dimostrazione della scalabilità orizzontale; `kitchen-service`, non ricevendo richieste, è definito senza `Service` associato; il frontend è esposto tramite un `Service` di tipo `NodePort` sulla porta 30080.

```
1 spec:
2   replicas: 2
3   template:
4     spec:
5       containers:
6         - name: order-service
7           image: mensa/order-service:1.0
8           imagePullPolicy: Never
9           envFrom:
10            - configMapRef: { name: mensa-config }
11           readinessProbe:
12             httpGet: { path: /healthz, port: 5002 }
```

Listing 3.4: Estratto del Deployment di `order-service`

La direttiva `imagePullPolicy: Never` è legata all'assenza di un registro di immagini in ambiente locale, questione affrontata nel paragrafo seguente.

3.6 Distribuzione delle immagini senza registro

In assenza di un registro, le immagini costruite sulla macchina di sviluppo non sono raggiungibili dai nodi del cluster. Lo script `load-images.sh` risolve il problema costruendo ciascuna immagine, esportandola in un archivio, trasferendola nelle macchine virtuali e importandola nel `containerd` locale:

```
1 docker build -t mensa/$s:1.0 ./$s
2 docker save mensa/$s:1.0 -o "$TMPDIR/$s.tar"
3 multipass transfer "$TMPDIR/$s.tar" $vm:/tmp/$s.tar
4 multipass exec $vm -- sudo ctr -n k8s.io images import /tmp/$s.tar
```

Listing 3.5: Caricamento manuale delle immagini nei nodi

Si tratta di una soluzione funzionale ma artigianale, che la seconda fase elimina introducendo un registro vero e proprio.

3.7 Verifica del deployment

Al termine della procedura il cluster risulta composto da tre nodi in stato `Ready` e i pod dell'applicazione in stato `Running`. L'applicazione risponde sulla porta 30080 di ciascun nodo.

La verifica funzionale consiste nell'inserire un ordine dall'interfaccia, osservare la comparsa dell'evento nei log di `kitchen-service` (a conferma del funzionamento della coda) e farne avanzare lo stato dal display di cucina fino alla rimozione dal tabellone Redis.

Capitolo 4

Fase 2: migrazione su Amazon Web Services

4.1 Architettura di destinazione

La seconda fase riproduce la stessa applicazione su AWS, sostituendo i componenti di supporto con i corrispondenti servizi gestiti e mantenendo Kubernetes come piattaforma di orchestrazione.

L'architettura, illustrata in Figura 4.1, comprende una VPC dedicata che ospita il cluster Kubernetes su tre istanze EC2, un Network Load Balancer come unico punto di ingresso pubblico, i servizi gestiti per la persistenza e la messaggistica, un registro privato per le immagini e i componenti di supporto all'automazione.

4.2 Scelta tra EKS e cluster autogestito

La traccia consentiva sia l'utilizzo di Amazon EKS sia l'installazione di un cluster Kubernetes su istanze EC2. È stata scelta la seconda opzione per tre motivi.

Il primo è economico: EKS comporta un costo fisso per il control plane indipendente dall'utilizzo effettivo, difficilmente giustificabile in un progetto didattico che viene acceso e spento ripetutamente. Il secondo è di continuità: il playbook Ansible sviluppato nella prima fase è riutilizzabile quasi integralmente, il che rende evidente il vantaggio di aver descritto la configurazione in modo dichiarativo e indipendente dall'infrastruttura sottostante. Il terzo è didattico: gestire direttamente il control plane comporta la comprensione di aspetti che EKS nasconde, come la pubblicazione dell'endpoint dell'API server e la gestione dei certificati.

Il contro di questa scelta va riconosciuto: il control plane è singolo e non replicato, e la sua manutenzione (aggiornamenti, backup di etcd) resta a carico dell'operatore. In un contesto di produzione la scelta sarebbe diversa.

4.3 Rete e sicurezza

La rete è costituita da una VPC con blocco di indirizzi 10.0.0.0/16 e due sottoreti pubbliche collocate in due Availability Zone distinte. La distribuzione su due zone non è opzionale: RDS ed ElastiCache richiedono un gruppo di sottoreti su almeno due zone, e costituisce inoltre il presupposto per l'alta disponibilità.

Si è scelto di non utilizzare un NAT Gateway. Tale componente comporterebbe un costo fisso mensile non trascurabile, e la sua funzione (consentire il traffico in uscita a istanze

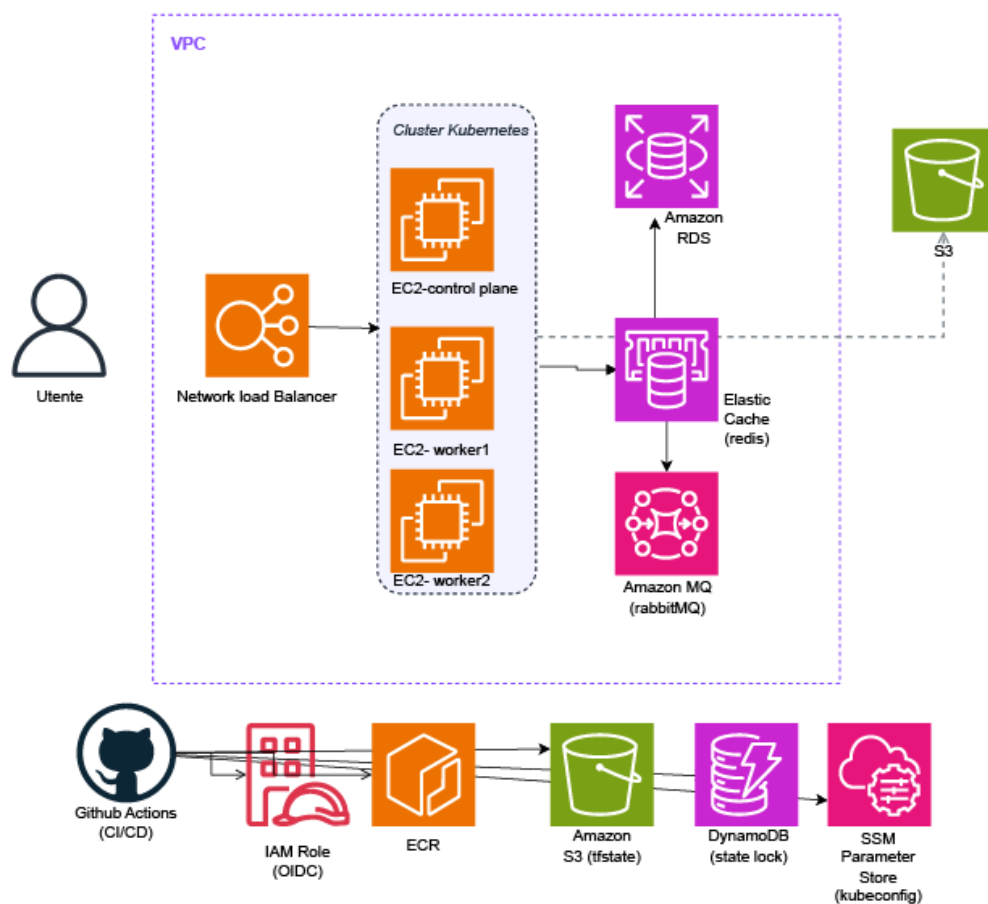


Figura 4.1: Architettura dell'applicazione su AWS

prive di indirizzo pubblico) non è necessaria in questo contesto: i nodi risiedono in sottoreti pubbliche e la loro protezione è affidata ai security group.

Sono definiti due security group con ruoli distinti. Il primo protegge i nodi del cluster: consente SSH e accesso all'API server esclusivamente dall'indirizzo IP dell'operatore, la porta 30080 per il traffico proveniente dal load balancer, e il traffico libero tra i nodi stessi, necessario alla comunicazione interna del cluster.

```

1 ingress {
2   description = "Traffico tra i nodi"
3   from_port   = 0
4   to_port     = 0
5   protocol    = "-1"
6   self        = true
7 }

```

Listing 4.1: Regola per il traffico interno al cluster

Il secondo security group protegge i servizi gestiti e presenta una caratteristica interessante: le regole non fanno riferimento a intervalli di indirizzi ma al security group dei nodi.

```

1 ingress {
2   description      = "PostgreSQL"
3   from_port        = 5432
4   to_port           = 5432
5   protocol          = "tcp"
6   security_groups   = [aws_security_group.nodes.id]
7 }

```

Listing 4.2: Accesso a PostgreSQL limitato ai soli nodi

Il controllo avviene quindi per *identità* e non per indirizzo: qualunque istanza appartenga al gruppo dei nodi può raggiungere il database, e nessun'altra. Poiché gli indirizzi delle istanze cambiano a ogni ricreazione, questa forma di regola è anche l'unica praticabile.

4.4 Servizi gestiti

Amazon RDS sostituisce il pod PostgreSQL. L'istanza è configurata con cifratura dello storage e, soprattutto, con `publicly_accessible` impostato a falso: il database non possiede un indirizzo pubblico ed è raggiungibile unicamente dall'interno della VPC.

Amazon ElastiCache sostituisce il pod Redis. La configurazione è volutamente minimale, senza repliche né backup: coerentemente con il ruolo assegnato a Redis nell'architettura, i dati contenuti sono ricostruibili.

Amazon MQ sostituisce il pod RabbitMQ. Questo componente ha richiesto due adattamenti. Il primo riguarda il dimensionamento: AWS non supporta più la classe `mq.t3.micro` con il motore RabbitMQ, riservandola ad ActiveMQ; è stata quindi utilizzata `mq.m7g.medium`, la classe più economica disponibile. Il secondo riguarda il protocollo: Amazon MQ espone esclusivamente AMQPS, ovvero AMQP incapsulato in TLS, sulla porta 5671 anziché la 5672 in chiaro. La libreria `pika` gestisce autonomamente gli URI con schema `amqps://`, per cui l'adattamento si è limitato alla stringa di connessione e all'apertura della porta corretta nel security group.

Amazon S3 sostituisce il volume delle immagini. È il punto in cui l'astrazione predisposta nella prima fase produce il suo effetto: impostando `STORAGE_BACKEND=s3` il ramo `boto3` del codice, fino a quel momento inutilizzato, inizia a scrivere sul bucket. A differenza del volume locale, il bucket è persistente e condiviso tra tutte le repliche del servizio.

4.5 Gestione delle credenziali

Nessuna credenziale è presente nei repository. Le password vengono generate da Terraform in fase di creazione:

```
1 resource "random_password" "db" {  
2   length   = 20  
3   special  = false  
4 }
```

Listing 4.3: Generazione della password del database

L'esclusione dei caratteri speciali evita problemi di interpretazione all'interno degli URI di connessione. Terraform compone quindi le stringhe di connessione complete, utilizzando le password generate e gli endpoint assegnati da AWS, e le memorizza cifrate su **SSM Parameter Store**. Lo script di deployment le rilegge da tale archivio e le trasforma in un Secret di Kubernetes.

La catena completa è quindi: Terraform genera e scrive su SSM, lo script legge da SSM e crea il Secret, Kubernetes inietta il Secret come variabili d'ambiente nei pod. In nessun punto della catena una credenziale transita per il repository.

Un discorso analogo riguarda l'accesso ad S3. Alle istanze EC2 è associato un ruolo IAM con i permessi minimi necessari: lettura dal registro delle immagini e scrittura limitata al solo bucket delle foto. La libreria `boto3` in esecuzione nel pod ottiene credenziali temporanee dal servizio di metadati dell'istanza, senza che alcuna chiave sia mai configurata.

4.6 Gestione dello stato di Terraform

Nella seconda fase lo stato di Terraform non risiede più sulla macchina locale ma su un bucket S3, con una tabella DynamoDB utilizzata come meccanismo di lock:

```
1 backend "s3" {  
2   bucket      = "mensa-tfstate-<account-id>"  
3   key         = "mensa/aws/terraform.tfstate"  
4   region     = "eu-central-1"  
5   dynamodb_table = "mensa-tfstate-lock"  
6   encrypt     = true  
7 }
```

Listing 4.4: Configurazione del backend remoto

Le motivazioni sono tre. Lo stato deve essere accessibile sia dalla postazione di sviluppo sia dalla pipeline automatica; deve essere versionato, perché la sua perdita comporterebbe l'impossibilità per Terraform di riconoscere le risorse già create, che resterebbero attive e fatturate; deve infine essere protetto da accessi concorrenti, poiché due esecuzioni simultanee di `apply` produrrebbero uno stato incoerente.

Il bucket e la tabella costituiscono un problema di circolarità: Terraform non può creare le risorse in cui salva il proprio stato. Sono quindi creati una sola volta tramite AWS CLI, operazione documentata nel README del progetto.

4.7 Registro delle immagini

Amazon ECR ospita le quattro immagini dell'applicazione. La definizione utilizza il costrutto `for_each` per evitare ripetizioni:

```
1 resource "aws_ecr_repository" "services" {  
2   for_each = toset(["menu-service", "order-service",  
3                     "kitchen-service", "frontend"])  
4   name      = "${var.project}/${each.key}"  
5   force_delete = true  
6 }
```

Listing 4.5: Creazione dei repository ECR

L'introduzione del registro elimina lo script di trasferimento manuale utilizzato nella prima fase: i nodi scaricano le immagini autonomamente, autorizzati dal ruolo IAM loro associato.

4.8 Bilanciamento del carico

Un Network Load Balancer distribuisce il traffico verso la porta 30080 dei nodi worker. La scelta di un bilanciatore di livello 4 anziché di livello 7 è motivata dalla semplicità del caso d'uso: non essendo necessario alcun instradamento basato su percorso o intestazioni HTTP, un Application Load Balancer introdurrebbe complessità e costi senza benefici.

4.9 Adattamenti al playbook Ansible

Il playbook della prima fase è stato riutilizzato con tre modifiche, tutte conseguenza del fatto che il cluster è ora raggiungibile da Internet.

L'API server pubblicizza l'indirizzo *privato*, poiché la comunicazione tra i nodi avviene all'interno della VPC. Tuttavia, per poter utilizzare `kubectl` dalla postazione di sviluppo, il certificato deve includere anche l'indirizzo *pubblico*, altrimenti la connessione verrebbe rifiutata per non corrispondenza del nome:

```
1 - name: kubeadm init  
2   command: >  
3     kubeadm init  
4     --pod-network-cidr=10.244.0.0/16  
5     --apiserver-advertise-address={{ ansible_default_ipv4.address }}  
6     --apiserver-cert-extra-sans={{ ansible_host }}
```

Listing 4.6: Inizializzazione con doppio indirizzo

Infine, il file `kubeconfig` scaricato contiene l'indirizzo privato e viene riscritto localmente con quello pubblico.

4.10 Pipeline di integrazione e distribuzione continua

La pipeline, realizzata con GitHub Actions, si attiva a ogni push sul ramo principale ed esegue quattro operazioni: autenticazione su AWS, costruzione delle immagini, caricamento su ECR e aggiornamento dei Deployment.

4.10.1 Autenticazione senza credenziali statiche

L'autenticazione utilizza OpenID Connect. GitHub emette un token firmato che attesta l'identità del workflow; AWS lo verifica e restituisce credenziali temporanee valide un'ora. Nessuna chiave di accesso permanente viene quindi memorizzata nei secret del repository.

La fiducia è stabilita tramite un identity provider e un ruolo IAM la cui policy di assunzione verifica il campo `sub` del token:

```
1 Condition = {
2   StringEquals = {
3     "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
4   }
5   StringLike = {
6     "token.actions.githubusercontent.com:sub" = [
7       "repo:${var.github_repo}:*",
8       "repo:vitomarino02-del@*/Cloud-Mensa-AWS-Version@*:*"
9     ]
10  }
11 }
```

Listing 4.7: Policy di assunzione del ruolo

La presenza di due modelli alternativi è la conseguenza di un problema affrontato durante lo sviluppo, descritto nel Paragrafo 5.2.6.

4.10.2 Esecuzione del deployment

L'aggiornamento del cluster presenta un ostacolo: l'API server accetta connessioni soltanto dall'indirizzo dell'operatore, mentre i runner di GitHub dispongono di indirizzi variabili e non prevedibili. Aprire l'API server al traffico proveniente da qualunque origine sarebbe stata la soluzione più immediata, ma avrebbe indebolito sensibilmente la sicurezza.

Si è scelto invece di utilizzare **SSM Run Command**: la pipeline non si collega al cluster, ma richiede ad AWS di eseguire un comando *all'interno* del control plane, dove `kubectl` opera in locale. Non viene aperta alcuna porta aggiuntiva e non circolano credenziali del cluster.

Le immagini sono etichettate con l'identificativo del commit anziché con una versione fissa. Questa scelta garantisce la tracciabilità: osservando i Deployment attivi è sempre possibile risalire esattamente alla revisione del codice in esecuzione.

4.11 Automazione dell'avvio

L'intera procedura di avvio è incapsulata nello script `up.sh`, che richiama in sequenza gli strumenti già descritti: rileva l'indirizzo IP corrente, crea l'infrastruttura, configura il cluster, costruisce e carica le immagini, distribuisce l'applicazione e ne popola il catalogo di immagini. Lo script `down.sh` esegue l'operazione inversa.

Si tratta di un orchestratore e non di una riscrittura: i singoli script restano utilizzabili autonomamente, il che consente ad esempio di aggiornare l'applicazione senza toccare l'infrastruttura.

Capitolo 5

Confronto, problemi affrontati e conclusioni

5.1 Corrispondenza tra le due fasi

La Tabella 5.1 riassume le corrispondenze tra i componenti delle due fasi.

Tabella 5.1: Corrispondenza tra deployment locale e deployment su AWS	
Fase 1 (locale)	Fase 2 (AWS)
Macchine virtuali Multipass	Istanze EC2
Pod PostgreSQL	Amazon RDS
Pod Redis	Amazon ElastiCache
Pod RabbitMQ	Amazon MQ (AMQPS, porta 5671)
Volume <code>emptyDir</code> per le immagini	Bucket Amazon S3
Trasferimento manuale delle immagini	Amazon ECR
Service NodePort 30080	NodePort dietro Network Load Balancer
Stato Terraform su file locale	Bucket S3 con lock su DynamoDB
Deployment manuale	GitHub Actions con OIDC

Il dato più significativo è ciò che *non* compare in tabella: il codice dei microservizi, che non ha subito alcuna modifica.

5.2 Problemi affrontati

Una parte rilevante del lavoro è consistita nella diagnosi di problemi non previsti. Se ne riportano i principali, con il procedimento seguito.

5.2.1 Concorrenza nella creazione dello schema

Al primo avvio dell'applicazione, con database vuoto, il servizio terminava con un errore di violazione di vincolo di unicità durante la creazione delle tabelle. La causa risiedeva nell'esecuzione concorrente: due worker del server applicativo, e in seguito due servizi distinti, eseguivano contemporaneamente la creazione dello schema sullo stesso database vuoto. Il primo completava l'operazione, il secondo falliva.

La soluzione adottata gestisce l'eccezione con un rollback e prosegue: chi arriva secondo trova lo schema già creato, condizione perfettamente accettabile.

5.2.2 Assenza di ordinamento all'avvio in Kubernetes

Il problema precedente si è ripresentato in forma diversa dopo il passaggio a Kubernetes. In Docker Compose la direttiva `depends_on` garantisce che i servizi applicativi vengano avviati soltanto dopo che i sistemi di persistenza rispondono. Kubernetes non prevede un meccanismo equivalente: i pod vengono avviati in parallelo e in ordine non deterministico.

Il servizio poteva quindi trovare il database non ancora disponibile. La gestione generica dell'eccezione introdotta in precedenza mascherava l'errore, lasciando il servizio in esecuzione ma privo di tabelle. La soluzione distingue le due condizioni: l'indisponibilità del database comporta un nuovo tentativo dopo alcuni secondi, la presenza di tabelle già create comporta la prosecuzione.

```
1 for _ in range(30):
2     try:
3         db.create_all()
4         break
5     except (IntegrityError, ProgrammingError):
6         db.session.rollback() # tabelle già create da un altro processo
7         break
8     except OperationalError:
9         db.session.rollback() # database non ancora pronto
10        time.sleep(2)
```

Listing 5.1: Attesa del database all'avvio

La lezione generale è che in un sistema distribuito la resilienza all'ordine di avvio deve risiedere nell'applicazione, non nell'orchestratore.

5.2.3 Schema condiviso tra servizi

Un terzo problema, emerso dallo stesso contesto, riguardava la tabella dei piatti, utilizzata in scrittura da menu-service e in sola lettura da order-service. Quest'ultimo ne dichiarava una versione ridotta, contenente i soli campi effettivamente utilizzati. Quando order-service veniva avviato per primo, creava una tabella incompleta, e le successive interrogazioni del menu fallivano per colonna inesistente.

La soluzione immediata è stata allineare le due dichiarazioni. La riflessione più generale è che la condivisione di tabelle tra microservizi costituisce un compromesso rispetto al modello puro, che prevede un database per servizio; è una scelta accettabile in un progetto di queste dimensioni, ma va riconosciuta come tale.

5.2.4 Vincoli dell'ambiente di sviluppo

La creazione delle macchine virtuali locali si è rivelata problematica. Multipass utilizzava VirtualBox come hypervisor, configurazione nella quale tutte le macchine ricevevano il medesimo indirizzo privato, non raggiungibile dall'esterno né utile alla comunicazione reciproca.

La diagnosi, condotta ispezionando le interfacce di rete all'interno di una macchina, ha portato a identificare la causa. Il passaggio a Hyper-V non era tuttavia percorribile, poiché l'edizione Home di Windows non lo include. La soluzione è consistita nello spostare l'intera catena di strumenti in WSL2, che espone il supporto alla virtualizzazione: Multipass utilizza in tale ambiente il driver QEMU. Va peraltro osservato che Ansible non è eseguibile nativamente su Windows, per cui il passaggio sarebbe stato comunque necessario.

5.2.5 Non determinismo nella selezione dell'immagine

Le istanze EC2 sono definite utilizzando un *data source* che seleziona l'immagine Ubuntu più recente pubblicata da Canonical. Questa impostazione introduce però un elemento di non determinismo: alla pubblicazione di una nuova immagine, l'esecuzione successiva di `apply` rileva una differenza e procede alla sostituzione delle istanze.

Il fenomeno si è verificato durante lo sviluppo, comportando la perdita del cluster e la necessità di rieseguire il playbook. Il rimedio consiste nel fissare l'identificativo dell'immagine o nell'escludere tale attributo dal confronto tramite un blocco `lifecycle`.

5.2.6 Identificatori immutabili nel token OIDC

Il problema più insidioso ha riguardato l'autenticazione della pipeline. Il tentativo di assunzione del ruolo falliva con un errore di autorizzazione, pur essendo il ruolo esistente, il `secret` configurato e la condizione apparentemente corretta.

La diagnosi è stata condotta per esclusione. Ampliando temporaneamente la condizione a qualunque repository, l'autenticazione riusciva: il problema non risiedeva quindi né nell'identity provider né nell'audience, ma nel confronto sul nome del repository. Restringendo la condizione al solo proprietario, l'autenticazione tornava a fallire, circostanza in apparente contraddizione con il nome effettivo del repository.

Si è reso quindi necessario ispezionare il contenuto reale del token. Un passo aggiuntivo nel workflow ne ha decodificato la sezione centrale, rivelando il valore effettivo del campo `sub`:

```
1 repo:vitomarino02-del@276350598/Cloud-Mensa-AWS-Version@1317345786:ref:refs/heads/main
```

Listing 5.2: Contenuto reale del claim

GitHub include ora, accanto ai nomi di proprietario e repository, i rispettivi identificatori numerici immutabili. La ragione è che i nomi possono essere modificati, gli identificatori no: l'inclusione di questi ultimi impedisce che una policy scritta per un repository continui a valere per un repository diverso che ne abbia ereditato il nome.

La condizione è stata quindi riscritta accettando entrambi i formati, mantenendo la restrizione al solo repository del progetto.

L'aspetto metodologicamente rilevante è che il problema non era diagnosticabile ispezionando la configurazione: la risposta era contenuta nel token, ed è stata ottenuta osservandone il contenuto effettivo anziché quello atteso.

5.2.7 Visibilità delle immagini su S3

Al termine della migrazione le foto dei piatti non venivano visualizzate: gli URI restituiti puntavano correttamente al bucket, ma questo era configurato senza accesso pubblico e restituiva un errore di autorizzazione.

Sono state valutate due soluzioni. La prima, adottata, consiste in una policy che consente la sola lettura degli oggetti; le chiavi sono identificatori casuali e la scrittura resta riservata al ruolo IAM dei nodi. La seconda, più restrittiva, avrebbe previsto la generazione di URL firmati a scadenza da parte del menu-service, mantenendo il bucket completamente privato: è la soluzione che si adotterebbe in un contesto reale con contenuti riservati.

5.3 Considerazioni sui costi

L'infrastruttura AWS comporta un costo dell'ordine di alcuni euro al giorno con tutti i componenti attivi. La possibilità di distruggere e ricreare l'intero ambiente tramite un singolo comando ha consentito di limitare la spesa all'effettivo tempo di utilizzo.

Questa possibilità è una diretta conseguenza dell'approccio Infrastructure as Code: l'infrastruttura, essendo interamente descritta in file di configurazione, diventa un artefatto riproducibile. Le uniche risorse mantenute attive sono il bucket dello stato e la tabella di lock, il cui costo è trascurabile.

5.4 Sviluppi possibili

Il progetto si presta a diverse estensioni.

L'installazione del *metrics-server* abiliterebbe la raccolta di metriche di utilizzo e, di conseguenza, l'*Horizontal Pod Autoscaler*, che consentirebbe di variare automaticamente il numero di repliche in funzione del carico.

Sul fronte dell'automazione, una seconda pipeline dedicata all'infrastruttura potrebbe eseguire `terraform plan` a ogni modifica dei file di configurazione, pubblicandone l'esito come commento, e riservare l'applicazione effettiva a un'approvazione manuale. La distinzione rispetto alla pipeline applicativa è motivata dall'asimmetria dei rischi: un errore applicativo si risolve con un rollback, un errore infrastrutturale può comportare la perdita di dati.

Sul piano dell'affidabilità, un control plane replicato su tre nodi eliminerebbe l'attuale punto singolo di guasto, e l'utilizzo di RDS in configurazione Multi-AZ garantirebbe la continuità del servizio dati.

Infine, la sostituzione di RabbitMQ con SQS e SNS, prevista tra le opzioni della traccia, consentirebbe di eliminare completamente la gestione del broker; tale modifica richiederebbe però l'adattamento del codice di produttore e consumatore, venendo meno il vantaggio dell'invarianza dell'applicazione.

5.5 Conclusioni

Il progetto ha realizzato un'applicazione a microservizi e ne ha verificato il deployment in due ambienti differenti, mantenendo invariato il codice applicativo. La proprietà è stata ottenuta esternalizzando integralmente la configurazione e introducendo un livello di astrazione sui componenti dipendenti dall'ambiente, come lo storage delle immagini.

L'infrastruttura è descritta come codice in entrambe le fasi, il che ne consente la ricreazione integrale e riproducibile. La configurazione dei nodi è affidata a un playbook idempotente riutilizzato in entrambi i contesti, a conferma dell'indipendenza degli strumenti di configuration management dall'infrastruttura sottostante.

La gestione delle credenziali è stata trattata come requisito e non come dettaglio: nessun segreto è presente nei repository, le password sono generate automaticamente e conservate cifrate, e l'autenticazione della pipeline avviene tramite credenziali temporanee.

I problemi incontrati, documentati nel presente capitolo, hanno costituito la parte più formativa del lavoro. In particolare, la diagnosi del problema sull'autenticazione OIDC ha richiesto di abbandonare l'ispezione della configurazione per osservare il dato effettivamente scambiato tra i sistemi: un metodo che si è rivelato risolutivo dove il ragionamento sulla configurazione attesa non lo era stato.

Appendice A

Listati principali

A.1 order-service

```
1  # order-service: gestione ordini
2  # Postgres = storico ordini, Redis = tabellone cucina (ordini attivi),
3  # RabbitMQ = eventi verso kitchen-service.
4  import os
5  import json
6  import time
7  import logging
8  from flask import Flask, request, jsonify
9  from flask_cors import CORS
10 from flask_sqlalchemy import SQLAlchemy
11 from sqlalchemy.exc import IntegrityError, ProgrammingError, OperationalError
12 import redis
13 import pika
14
15 db = SQLAlchemy()
16 log = logging.getLogger("order-service")
17
18 REDIS_URL = os.environ.get("REDIS_URL", "redis://localhost:6379/0")
19 RABBITMQ_URL = os.environ.get("RABBITMQ_URL",
20                               "amqp://guest:guest@localhost:5672/")
21 EXCHANGE = "order.events"      # exchange fanout: il messaggio arriva a tutti
22                                # i consumer
23 BOARD_KEY = "kitchen:board"    # hash Redis: id ordine -> json
24
25 STATES = ["ricevuto", "in_preparazione", "pronto", "ritirato"]
26
27 class Dish(db.Model):
28     """Stessa tabella del menu-service, qui usata in sola lettura.
29     Lo schema DEVE essere identico a quello del menu-service: in K8s l'ordine
30     di avvio e' casuale e la tabella la crea il primo servizio che parte."""
31     __tablename__ = "dishes"
32     id = db.Column(db.Integer, primary_key=True)
33     name = db.Column(db.String(120), nullable=False)
34     description = db.Column(db.String(255))
35     price = db.Column(db.Numeric(6, 2), nullable=False, default=0)
36     category = db.Column(db.String(40), default="primo")
37     image_key = db.Column(db.String(200))
38     available = db.Column(db.Boolean, default=True)
39
40 class Order(db.Model):
41     __tablename__ = "orders"
```

```

42     id = db.Column(db.Integer, primary_key=True)
43     customer = db.Column(db.String(120), nullable=False)
44     status = db.Column(db.String(20), nullable=False, default="ricevuto")
45     total = db.Column(db.Numeric(7, 2), nullable=False, default=0)
46     created_at = db.Column(db.DateTime, server_default=db.func.now())
47     items = db.relationship("OrderItem", backref="order",
48                             cascade="all, delete-orphan")
49
50     def to_dict(self):
51         return {"id": self.id, "customer": self.customer, "status":
52                 self.status,
53                 "total": float(self.total),
54                 "items": [i.to_dict() for i in self.items]}
55
56 class OrderItem(db.Model):
57     __tablename__ = "order_items"
58     id = db.Column(db.Integer, primary_key=True)
59     order_id = db.Column(db.Integer, db.ForeignKey("orders.id"),
60                          nullable=False)
61     dish_id = db.Column(db.Integer, db.ForeignKey("dishes.id"),
62                          nullable=False)
63     dish_name = db.Column(db.String(120)) # nome e prezzo copiati: lo
64                                           storico
65     qty = db.Column(db.Integer, nullable=False, default=1)
66     unit_price = db.Column(db.Numeric(6, 2), nullable=False, default=0) #
67                                           resta giusto anche se il menu cambia
68
69     def to_dict(self):
70         return {"dish_id": self.dish_id, "dish_name": self.dish_name,
71                 "qty": self.qty, "unit_price": float(self.unit_price)}
72
73 # --- tabellone su Redis: se Redis non risponde l'app continua a funzionare
74 # ---
75
76 _redis = None
77
78 def r():
79     global _redis
80     if _redis is None:
81         _redis = redis.from_url(REDIS_URL, decode_responses=True)
82     return _redis
83
84 def board_put(order):
85     try:
86         r().hset(BOARD_KEY, order["id"], json.dumps(order))
87     except Exception as e:
88         log.warning("Redis non disponibile: %s", e)
89
90 def board_remove(order_id):
91     try:
92         r().hdel(BOARD_KEY, order_id)
93     except Exception as e:
94         log.warning("Redis non disponibile: %s", e)
95
96 def board_all():
97     try:
98         return [json.loads(v) for v in r().hvals(BOARD_KEY)]

```

```

98     except Exception as e:
99         log.warning("Redis non disponibile: %s", e)
100         return []
101
102
103 # --- eventi su RabbitMQ ---
104
105 def publish(event_type, payload):
106     body = json.dumps({"type": event_type, "data": payload})
107     try:
108         conn = pika.BlockingConnection(pika.URLParameters(RABBITMQ_URL))
109         ch = conn.channel()
110         ch.exchange_declare(exchange=EXCHANGE, exchange_type="fanout",
111                             durable=True)
112         ch.basic_publish(exchange=EXCHANGE, routing_key="", body=body)
113         conn.close()
114     except Exception as e:
115         log.warning("RabbitMQ non disponibile: %s", e)
116
117 def create_app():
118     app = Flask(__name__)
119     CORS(app)
120     app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
121         "DATABASE_URL", "postgresql://mensa:mensa@localhost:5432/mensa")
122     db.init_app(app)
123
124     # DB non pronto -> riprova; tabelle gia' create dal menu-service ->
125     # prosegui
126     with app.app_context():
127         for _ in range(30):
128             try:
129                 db.create_all()
130                 break
131             except (IntegrityError, ProgrammingError):
132                 db.session.rollback()
133                 break
134             except OperationalError:
135                 db.session.rollback()
136                 time.sleep(2)
137
138 @app.get("/healthz")
139 def healthz():
140     return {"status": "ok", "service": "order-service"}
141
142 @app.post("/api/orders")
143 def create_order():
144     d = request.get_json(force=True)
145     customer = d.get("customer", "").strip()
146     items = d.get("items", [])
147     if not customer or not items:
148         return {"error": "cliente e items obbligatori"}, 400
149     order = Order(customer=customer, status="ricevuto", total=0)
150     db.session.add(order)
151     total = 0
152     for it in items:
153         dish = db.session.get(Dish, it["dish_id"])
154         if not dish:
155             return {"error": f"piatto {it['dish_id']} inesistente"}, 400
156         qty = int(it.get("qty", 1))
157         total += float(dish.price) * qty # totale calcolato lato server

```



```

157         order.items.append(OrderItem(dish_id=dish.id,
158                                     dish_name=dish.name,
159                                     qty=qty, unit_price=dish.price))
160     order.total = total
161     db.session.commit()
162     board_put(order.to_dict())          # lavagna cucina
163     publish("order.created", order.to_dict()) # campanello
164     return jsonify(order.to_dict()), 201
165
166 @app.get("/api/orders")
167 def list_orders():
168     # ?active=1 -> tabellone veloce da Redis; senza -> storico da
169     # Postgres
170     if request.args.get("active"):
171         board = board_all()
172         board.sort(key=lambda o: o["id"])
173         return jsonify(board)
174     orders = Order.query.order_by(Order.id.desc()).limit(50).all()
175     return jsonify([o.to_dict() for o in orders])
176
177 @app.get("/api/orders/<int:oid>")
178 def get_order(oid):
179     return jsonify(Order.query.get_or_404(oid).to_dict())
180
181 @app.post("/api/orders/<int:oid>/advance")
182 def advance(oid):
183     order = Order.query.get_or_404(oid)
184     i = STATES.index(order.status)
185     if i >= len(STATES) - 1:
186         return {"error": "ordine gia' completato"}, 409
187     order.status = STATES[i + 1]
188     db.session.commit()
189     if order.status == "ritirato":
190         board_remove(order.id) # via dalla lavagna, resta nello storico
191     else:
192         board_put(order.to_dict())
193     if order.status == "pronto":
194         publish("order.ready", order.to_dict())
195     return jsonify(order.to_dict())
196
197 return app
198
199 app = create_app()
200
201 if __name__ == "__main__":
202     app.run(host="0.0.0.0", port=5002)

```

Listing A.1: order-service/app.py

A.2 menu-service

```

1 # menu-service: catalogo piatti + immagini
2 # Le foto vanno su volume locale (Fase 1) o su S3 (Fase 2), scelta con
3 # STORAGE_BACKEND.
4 import os
5 import uuid
6 import io
7 import time
8 from flask import Flask, request, jsonify, send_file

```

```

8 from flask_cors import CORS
9 from flask_sqlalchemy import SQLAlchemy
10 from sqlalchemy.exc import IntegrityError, ProgrammingError, OperationalError
11
12 db = SQLAlchemy()
13
14 STORAGE_BACKEND = os.environ.get("STORAGE_BACKEND", "local") # local / s3
15 IMAGE_DIR = os.environ.get("IMAGE_DIR", "/data/images")
16
17
18 class Dish(db.Model):
19     __tablename__ = "dishes"
20     id = db.Column(db.Integer, primary_key=True)
21     name = db.Column(db.String(120), nullable=False)
22     description = db.Column(db.String(255))
23     price = db.Column(db.Numeric(6, 2), nullable=False, default=0)
24     category = db.Column(db.String(40), default="primo") #
25         primo/secondo/contorno/bevanda/dolce
26     image_key = db.Column(db.String(200))
27     available = db.Column(db.Boolean, default=True)
28
29     def to_dict(self):
30         return {
31             "id": self.id, "name": self.name, "description":
32                 self.description,
33             "price": float(self.price), "category": self.category,
34             "available": self.available,
35             "image_url": image_url(self.image_key) if self.image_key else
36                 None,
37         }
38
39 # --- storage immagini: stesse funzioni, backend diverso in base
40 # all'ambiente ---
41
42 def save_image(data, content_type):
43     ext = ".png" if "png" in content_type else ".jpg"
44     key = uuid.uuid4().hex + ext
45     if STORAGE_BACKEND == "s3":
46         import boto3
47         s3 = boto3.client("s3", region_name=os.environ.get("AWS_REGION",
48             "us-east-1"))
49         s3.put_object(Bucket=os.environ["S3_BUCKET"], Key=key, Body=data,
50             ContentType=content_type)
51     else:
52         os.makedirs(IMAGE_DIR, exist_ok=True)
53         with open(os.path.join(IMAGE_DIR, key), "wb") as f:
54             f.write(data)
55     return key
56
57 def image_url(key):
58     if STORAGE_BACKEND == "s3":
59         bucket = os.environ["S3_BUCKET"]
60         region = os.environ.get("AWS_REGION", "us-east-1")
61         return f"https://{bucket}.s3.{region}.amazonaws.com/{key}"
62     return f"/api/menu/images/{key}" # servita da questo stesso servizio
63
64 SEED = [
65     ("Pasta alla Norma", "Pomodoro, melanzane e ricotta salata", 4.50,
66         "primo"),

```

```

64     ("Risotto ai funghi", "Riso carnaroli e funghi porcini", 5.00, "primo"),
65     ("Cotoletta alla milanese", "Con patatine fritte", 6.00, "secondo"),
66     ("Pollo alla griglia", "Petto di pollo con insalata", 5.50, "secondo"),
67     ("Insalata mista", "Verdure fresche di stagione", 3.00, "contorno"),
68     ("Patine fritte", "Porzione media", 2.50, "contorno"),
69     ("Acqua naturale 0.5L", "Bottiglia", 0.50, "bevanda"),
70     ("Coca-Cola 0.33L", "Lattina", 1.50, "bevanda"),
71     ("Tiramisu", "Fatto in casa", 3.00, "dolce"),
72 ]
73
74
75 def create_app():
76     app = Flask(__name__)
77     CORS(app)
78     app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
79         "DATABASE_URL", "postgresql://mensa:mensa@localhost:5432/mensa")
80     app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024 # upload max 5 MB
81     db.init_app(app)
82
83     # In Kubernetes non esiste depends_on: il DB potrebbe non essere ancora
84     # pronto, quindi si riprova. Se invece un altro worker ha gia' creato
85     # tabelle e seed (race), si prosegue.
86     with app.app_context():
87         for _ in range(30):
88             try:
89                 db.create_all()
90                 if Dish.query.count() == 0:
91                     for name, desc, price, cat in SEED:
92                         db.session.add(Dish(name=name, description=desc,
93                                             price=price, category=cat))
94                     db.session.commit()
95                     break
96             except (IntegrityError, ProgrammingError):
97                 db.session.rollback() # race: ha gia' fatto tutto un altro
98                 break
99             except OperationalError:
100                 db.session.rollback() # DB non pronto: aspetta e riprova
101                 time.sleep(2)
102
103 @app.get("/healthz")
104 def healthz():
105     return {"status": "ok", "service": "menu-service"}
106
107 @app.get("/api/menu")
108 def list_menu():
109     dishes = Dish.query.filter_by(available=True).order_by(Dish.id).all()
110     return jsonify([d.to_dict() for d in dishes])
111
112 @app.post("/api/menu/<int:dish_id>/image")
113 def upload_image(dish_id):
114     dish = Dish.query.get_or_404(dish_id)
115     f = request.files.get("file")
116     if not f:
117         return {"error": "file mancante"}, 400
118     dish.image_key = save_image(f.read(), f.content_type or "image/jpeg")
119     db.session.commit()
120     return dish.to_dict(), 201
121
122 @app.get("/api/menu/images/<key>")
123 def serve_image(key):
124     # usata solo con storage locale (con S3 l'URL punta direttamente al
125     bucket)

```

```

125     path = os.path.join(IMAGE_DIR, os.path.basename(key))
126     if not os.path.exists(path):
127         return {"error": "immagine non trovata"}, 404
128     mime = "image/png" if key.endswith(".png") else "image/jpeg"
129     return send_file(path, mimetype=mime)
130
131     return app
132
133
134 app = create_app()
135
136 if __name__ == "__main__":
137     app.run(host="0.0.0.0", port=5001)

```

Listing A.2: menu-service/app.py

A.3 kitchen-service

```

1  # kitchen-service: consuma gli eventi ordine da RabbitMQ
2  # In locale "notifica" = riga di log; in Fase 2 (AWS) al suo posto ci sarà
   SNS.
3  import os
4  import json
5  import time
6  import logging
7  import pika
8
9  logging.basicConfig(level=logging.INFO,
10                     format="%(asctime)s %(levelname)s %(message)s")
11  log = logging.getLogger("kitchen")
12
13  RABBITMQ_URL = os.environ.get("RABBITMQ_URL",
14                               "amqp://guest:guest@localhost:5672/")
15  EXCHANGE = "order.events"
16
17  def on_message(ch, method, properties, body):
18      try:
19          ev = json.loads(body)
20      except Exception:
21          ev = {"raw": body.decode(errors="replace")}
22      t, data = ev.get("type"), ev.get("data", {})
23      if t == "order.created":
24          log.info("NUOVO ORDINE #s per %s - %s articoli (tot %.2f EUR)",
25                  data.get("id"), data.get("customer"),
26                  len(data.get("items", [])), data.get("total", 0))
27      elif t == "order.ready":
28          log.info("ORDINE #s PRONTO -> avvisa %s", data.get("id"),
29                  data.get("customer"))
30      else:
31          log.info("evento %s: %s", t, data)
32
33
34  def main():
35      #Inserito per effettuare una prova ogni 5 secondi se RABBITMQ non è
       pronto
36      while True:
37          try:
38              conn = pika.BlockingConnection(pika.URLParameters(RABBITMQ_URL))
39              ch = conn.channel()

```

```

40         ch.exchange_declare(exchange=EXCHANGE, exchange_type="fanout",
41                             durable=True)
42         q = ch.queue_declare(queue="kitchen", durable=True)
43         ch.queue_bind(exchange=EXCHANGE, queue=q.method.queue)
44         log.info("In ascolto sull'exchange '%s'...", EXCHANGE)
45         ch.basic_consume(queue=q.method.queue,
46                         on_message_callback=on_message, auto_ack=True)
47         ch.start_consuming()
48     except pika.exceptions.AMQPConnectionError:
49         log.warning("RabbitMQ non pronto, ritento fra 5s...")
50         time.sleep(5)
51     except KeyboardInterrupt:
52         break
53
54
55 if __name__ == "__main__":
56     main()

```

Listing A.3: kitchen-service/consumer.py

A.4 Infrastruttura locale

```

1  # Fase 1 - Infrastruttura locale: 3 VM Ubuntu create con Multipass.
2  # 1 control-plane + 2 worker per il cluster Kubernetes (kubeadm via Ansible).
3  # Uso: terraform init && terraform apply
4
5  terraform {
6      required_version = ">= 1.6"
7      required_providers {
8          multipass = {
9              source = "larstobi/multipass"
10             version = "~> 1.4"
11         }
12         local = {
13             source = "hashicorp/local"
14             version = "~> 2.5"
15         }
16     }
17 }
18
19 provider "multipass" {}
20
21 variable "worker_count" {
22     description = "Numero di nodi worker"
23     type        = number
24     default     = 2
25 }
26
27 # kubeadm richiede almeno 2 CPU e 2GB di RAM per nodo
28 variable "cpus" {
29     type = number
30     default = 2
31 }
32
33 variable "memory" {
34     type = string
35     default = "2G"
36 }
37
38 variable "disk" {

```

```

39     type      = string
40     default   = "10G"
41 }
42
43 variable "ssh_public_key_path" {
44     type      = string
45     default   = "~/.ssh/id_rsa.pub"
46 }
47
48 variable "ssh_private_key_path" {
49     type      = string
50     default   = "~/.ssh/id_rsa"
51 }
52
53 # cloud-init: primo avvio delle VM (utente + chiave SSH per Ansible + docker)
54 resource "local_file" "cloudinit" {
55     filename = "${path.module}/.cloud-init/generated.yaml"
56     content = templatefile("${path.module}/cloud-init.yaml.tftpl", {
57         ssh_public_key = trimspace(file(pathexpand(var.ssh_public_key_path)))
58     })
59 }
60
61 # ---- Control plane ----
62 resource "multipass_instance" "control_plane" {
63     name      = "mensa-cp"
64     image     = "22.04"
65     cpus      = var.cpus
66     memory    = var.memory
67     disk      = var.disk
68     cloudinit_file = local_file.cloudinit.filename
69 }
70
71 # ---- Workers ----
72 resource "multipass_instance" "worker" {
73     count     = var.worker_count
74     name      = "mensa-worker-${count.index + 1}"
75     image     = "22.04"
76     cpus      = var.cpus
77     memory    = var.memory
78     disk      = var.disk
79     cloudinit_file = local_file.cloudinit.filename
80 }
81
82 # Inventory per Ansible generato con gli IP reali delle VM
83 resource "local_file" "ansible_inventory" {
84     filename = "${path.module}/ansible/inventory.ini"
85     content = templatefile("${path.module}/inventory.tpl", {
86         cp_ip      = multipass_instance.control_plane.ipv4
87         worker_ips = [for w in multipass_instance.worker : w.ipv4]
88         ssh_key_path = pathexpand(var.ssh_private_key_path)
89     })
90 }
91
92 output "control_plane_ip" {
93     value = multipass_instance.control_plane.ipv4
94 }
95
96 output "worker_ips" {
97     value = [for w in multipass_instance.worker : w.ipv4]
98 }
99
100 output "frontend_url" {

```

```

101 description = "URL dell'app dopo il deploy su K8s (NodePort 30080)"
102 value       = "http://${multipass_instance.worker[0].ipv4}:30080"
103 }

```

Listing A.4: locale/main.tf

A.5 Playbook Ansible

```

1  # Cluster Kubernetes (kubeadm) sulle istanze EC2 create da Terraform.
2  # Stesso playbook della Fase 1, con due differenze dovute ad AWS:
3  # - l'API server pubblicizza l'IP privato ma il certificato include anche
4  #   quello pubblico, per poter usare kubectl dal PC
5  # - il kubeconfig scaricato viene riscritto con l'IP pubblico
6  # Uso: ansible-playbook -i inventory.ini site.yml
7
8  - name: Prerequisiti comuni a tutti i nodi
9    hosts: k8s_cluster
10   become: true
11   tasks:
12     - name: Disattiva la swap
13       command: swapoff -a
14       changed_when: false
15
16     - name: Moduli kernel per container e rete bridge
17       copy:
18         dest: /etc/modules-load.d/k8s.conf
19         content: |
20             overlay
21             br_netfilter
22
23     - name: Carica subito i moduli
24       command: modprobe {{ item }}
25       loop: [overlay, br_netfilter]
26       changed_when: false
27
28     - name: Parametri di rete richiesti da Kubernetes
29       copy:
30         dest: /etc/sysctl.d/k8s.conf
31         content: |
32             net.bridge.bridge-nf-call-iptables = 1
33             net.bridge.bridge-nf-call-ip6tables = 1
34             net.ipv4.ip_forward = 1
35       notify: applica sysctl
36
37     - name: Installa containerd
38       apt:
39         name: containerd
40         state: present
41         update_cache: true
42
43     - name: Configura containerd con cgroup systemd
44       shell: |
45         mkdir -p /etc/containerd
46         containerd config default | sed 's/SystemdCgroup =
47             false/SystemdCgroup = true/' > /etc/containerd/config.toml
48       args:
49         creates: /etc/containerd/config.toml
50       notify: riavvia containerd
51
52     - name: Chiave APT del repository Kubernetes

```

```

52     shell: |
53         mkdir -p /etc/apt/keyrings
54         curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key |
55             gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
56     args:
57         creates: /etc/apt/keyrings/kubernetes-apt-keyring.gpg
58
59 - name: Repository Kubernetes
60   apt_repository:
61     repo: "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
62           https://pkgs.k8s.io/core:/stable:/v1.30/deb/ #"
63     state: present
64
65 - name: Installa kubelet, kubeadm e kubectl
66   apt:
67     name: [kubelet, kubeadm, kubectl]
68     state: present
69     update_cache: true
70
71 - name: Blocca gli aggiornamenti automatici dei pacchetti K8s
72   command: apt-mark hold kubelet kubeadm kubectl
73   changed_when: false
74
75 # serve al control plane per rigenerare il token ECR durante il deploy
76 # automatico lanciato da GitHub Actions via SSM
77
78 - name: Installa la AWS CLI
79   snap:
80     name: aws-cli
81     classic: true
82
83 handlers:
84
85 - name: applica sysctl
86   command: sysctl --system
87
88 - name: riavvia containerd
89   systemd:
90     name: containerd
91     state: restarted
92     enabled: true
93
94 - name: Inizializza il control plane
95   hosts: control_plane
96   become: true
97   tasks:
98     # advertise-address: IP privato (comunicazione interna al cluster)
99     # cert-extra-sans: IP pubblico (per usare kubectl dal PC)
100
101 - name: kubeadm init
102   command: >
103     kubeadm init
104     --pod-network-cidr=10.244.0.0/16
105     --apiserver-advertise-address={{ ansible_default_ipv4.address }}
106     --apiserver-cert-extra-sans={{ ansible_host }}
107   args:
108     creates: /etc/kubernetes/admin.conf
109
110 - name: kubeconfig per l'utente ubuntu
111   shell: |
112     mkdir -p /home/ubuntu/.kube
113     cp /etc/kubernetes/admin.conf /home/ubuntu/.kube/config
114     chown -R ubuntu:ubuntu /home/ubuntu/.kube
115   args:
116     creates: /home/ubuntu/.kube/config

```



```

112 - name: Installa la rete dei pod (Flannel)
113   command: kubectl apply -f
114     https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
115   environment:
116     KUBECONFIG: /etc/kubernetes/admin.conf
117   changed_when: false
118
119 - name: Genera il comando di join per i worker
120   command: kubeadm token create --print-join-command
121   register: join_cmd
122   changed_when: false
123
124 - name: Scarica il kubeconfig sul PC
125   fetch:
126     src: /etc/kubernetes/admin.conf
127     dest: "{{ playbook_dir }}/kubeconfig"
128     flat: true
129
130 - name: Unisci i worker al cluster
131   hosts: workers
132   become: true
133   tasks:
134     - name: kubeadm join
135       command: "{{ hostvars[groups['control_plane'][0]].join_cmd.stdout }}"
136       args:
137         creates: /etc/kubernetes/kubelet.conf
138
139 - name: Prepara il kubeconfig locale
140   hosts: localhost
141   connection: local
142   gather_facts: false
143   tasks:
144     # Nel file scaricato l'API server e' l'IP privato: da fuori non e'
145     # raggiungibile, quindi va sostituito con quello pubblico.
146     - name: Punta il kubeconfig all'IP pubblico del control plane
147       replace:
148         path: "{{ playbook_dir }}/kubeconfig"
149         regexp: 'server: https://.*:6443'
150         replace: "server: https://{{
           hostvars[groups['control_plane'][0]].ansible_host }}:6443"

```

Listing A.5: ansible/site.yml

A.6 Infrastruttura AWS

```

1
2 # VPC ,cluster Kubernetes su EC2 , servizi gestiti
3
4
5 terraform {
6   required_version = ">= 1.6"
7
8   required_providers {
9     aws = {
10       source = "hashicorp/aws"
11       version = "~> 5.60"
12     }
13     local = {
14       source = "hashicorp/local"

```

```

15     version = "~> 2.5"
16   }
17   random = {
18     source = "hashicorp/random"
19     version = "~> 3.6"
20   }
21 }
22
23 # Stato remoto, S3 lo conserva, DynamoDB impedisce apply simultanei
24 backend "s3" {
25   bucket      = "mensa-tfstate-862087104689"
26   key         = "mensa/aws/terraform.tfstate"
27   region      = "eu-central-1"
28   dynamodb_table = "mensa-tfstate-lock"
29   encrypt     = true
30 }
31 }
32
33 provider "aws" {
34   region = var.region
35
36   default_tags {
37     tags = {
38       Project = "cloud-mensa"
39       Env     = "aws"
40     }
41   }
42 }
43
44 # ----- variabili
45
46 variable "region" {
47   type      = string
48   default   = "eu-central-1"
49 }
50
51 variable "project" {
52   type      = string
53   default   = "mensa"
54 }
55
56 variable "vpc_cidr" {
57   type      = string
58   default   = "10.0.0.0/16"
59 }
60
61 # Due subnet pubbliche in AZ diverse: RDS e i load balancer richiedono
almeno due Availability Zone.
62 variable "public_subnet_cidrs" {
63   type      = list(string)
64   default   = ["10.0.1.0/24", "10.0.2.0/24"]
65 }
66
67 # Indirizzo da cui si collega in SSH e all'app
68 variable "my_ip_cidr" {
69   type      = string
70   description = "IP pubblico autorizzato per SSH"
71 }
72
73 variable "instance_type" {
74   type      = string
75   default   = "t3.small" # kubeadm richiede 2 vCPU / 2 GB

```

```

76 }
77
78 variable "worker_count" {
79     type      = number
80     default   = 2
81 }
82
83 variable "ssh_public_key_path" {
84     type      = string
85     default   = "~/.ssh/id_rsa.pub"
86 }
87
88 # ----- rete
89 # VPC dedicata con due subnet pubbliche.
90 # Niente NAT Gateway, i nodi stanno in subnet pubbliche e protetti dai
91 #senza la vpc rds e ElastiCache non darebbero un endpoint DNS risolvibile
92
93 data "aws_availability_zones" "available" {
94     state = "available"
95 }
96
97 resource "aws_vpc" "main" {
98     cidr_block           = var.vpc_cidr
99     enable_dns_hostnames = true
100    enable_dns_support   = true
101
102    tags = { Name = "${var.project}-vpc" }
103 }
104 #INTERNET GATEWAY, serve per la rotta della vpc verso internet
105 resource "aws_internet_gateway" "main" {
106     vpc_id = aws_vpc.main.id
107     tags   = { Name = "${var.project}-igw" }
108 }
109
110 resource "aws_subnet" "public" { #una subnet creata per ogni cidr
111     count                = length(var.public_subnet_cidrs)
112     vpc_id               = aws_vpc.main.id
113     cidr_block           = var.public_subnet_cidrs[count.index]
114     availability_zone     =
115         data.aws_availability_zones.available.names[count.index]
116     map_public_ip_on_launch = true
117
118     tags = { Name = "${var.project}-public-${count.index + 1}" }
119 }
120
121 resource "aws_route_table" "public" {
122     vpc_id = aws_vpc.main.id
123
124     route {
125         cidr_block = "0.0.0.0/0"
126         gateway_id = aws_internet_gateway.main.id
127     }
128
129     tags = { Name = "${var.project}-public-rt" }
130 }
131
132 resource "aws_route_table_association" "public" {
133     count                = length(aws_subnet.public)
134     subnet_id           = aws_subnet.public[count.index].id
135     route_table_id      = aws_route_table.public.id
136 }

```

```

136
137 # ---- Security group dei nodi del cluster ----
138 resource "aws_security_group" "nodes" {
139     name           = "${var.project}-nodes"
140     description    = "Nodi Kubernetes"
141     vpc_id         = aws_vpc.main.id
142
143     # SSH solo dal proprio IP (per Ansible)
144     ingress {
145         description = "SSH"
146         from_port   = 22
147         to_port     = 22
148         protocol    = "tcp"
149         cidr_blocks = [var.my_ip_cidr]
150     }
151
152     # API server Kubernetes
153     ingress {
154         description = "kube-apiserver"
155         from_port   = 6443
156         to_port     = 6443
157         protocol    = "tcp"
158         cidr_blocks = [var.my_ip_cidr]
159     }
160
161     # NodePort dell'app raggiunta dal load balancer e dal proprio IP
162     ingress {
163         description = "NodePort frontend"
164         from_port   = 30080
165         to_port     = 30080
166         protocol    = "tcp"
167         cidr_blocks = [var.vpc_cidr, var.my_ip_cidr]
168     }
169
170     # Traffico interno al cluster
171     ingress {
172         description = "Traffico tra i nodi"
173         from_port   = 0
174         to_port     = 0
175         protocol    = "-1"
176         self        = true
177     }
178
179     egress {
180         from_port   = 0
181         to_port     = 0
182         protocol    = "-1"
183         cidr_blocks = ["0.0.0.0/0"]
184     }
185
186     tags = { Name = "${var.project}-nodes-sg" }
187 }
188
189 # ---- Security group dei servizi gestiti: accessibili solo dai nodi-
190 resource "aws_security_group" "data" {
191     name           = "${var.project}-data"
192     description    = "RDS, ElastiCache, Amazon MQ"
193     vpc_id         = aws_vpc.main.id
194
195     ingress {
196         description = "PostgreSQL"
197         from_port   = 5432

```

```

198     to_port      = 5432
199     protocol     = "tcp"
200     security_groups = [aws_security_group.nodes.id] #possono connettersi a
        Postgres solo le risorse che appartengono al security group dei nodi
201 }
202
203 ingress {
204     description     = "Redis"
205     from_port       = 6379
206     to_port         = 6379
207     protocol        = "tcp"
208     security_groups = [aws_security_group.nodes.id]
209 }
210
211 ingress {
212     description     = "AMQP (RabbitMQ)"
213     from_port       = 5671
214     to_port         = 5671
215     protocol        = "tcp"
216     security_groups = [aws_security_group.nodes.id]
217 }
218
219 egress {
220     from_port = 0
221     to_port   = 0
222     protocol  = "-1"
223     cidr_blocks = ["0.0.0.0/0"]
224 }
225
226 tags = { Name = "${var.project}-data-sg" }
227 }
228
229 # ----- dati
230 # Bucket S3 per le foto dei piatti: sostituisce il volume locale della Fase 1
231 # nel codice basta STORAGE_BACKEND=s3, nessuna modifica applicativa.
232
233 resource "aws_s3_bucket" "images" {
234     bucket =
        "${var.project}-images-${data.aws_caller_identity.current.account_id}"
235     force_destroy = true
236 }
237
238 resource "aws_s3_bucket_public_access_block" "images" {
239     bucket = aws_s3_bucket.images.id
240
241     block_public_acls       = true
242     block_public_policy     = false
243     ignore_public_acls     = true
244     restrict_public_buckets = false
245 }
246
247 # Password generate da Terraform, non scritte nel codice.
248 # special = false è stato inserito per non rompere il parsing delle URL di
    connessione
249 resource "random_password" "db" {
250     length = 20
251     special = false
252 }
253
254 resource "random_password" "mq" {
255     length = 20
256     special = false # Amazon MQ non ammette virgole, due punti e spazi

```

```

257 }
258
259 # RDS ed ElastiCache vanno in un gruppo di subnet su due AZ diverse
260 resource "aws_db_subnet_group" "main" {
261     name           = "${var.project}-db-subnets"
262     subnet_ids     = aws_subnet.public[*].id
263 }
264
265 resource "aws_elasticache_subnet_group" "main" {
266     name           = "${var.project}-cache-subnets"
267     subnet_ids     = aws_subnet.public[*].id
268 }
269
270 # ---- RDS PostgreSQL: qui sostituisce il pod postgres della fase locale ----
271 resource "aws_db_instance" "postgres" {
272     identifier      = "${var.project}-db"
273     engine          = "postgres"
274     engine_version  = "16"
275     instance_class  = "db.t3.micro" # free tier
276
277     allocated_storage = 20
278     storage_encrypted = true
279
280     db_name      = "mensa"
281     username     = "mensa"
282     password     = random_password.db.result
283
284     db_subnet_group_name = aws_db_subnet_group.main.name
285     vpc_security_group_ids = [aws_security_group.data.id]
286     publicly_accessible   = false # raggiungibile solo dentro la VPC
287
288     skip_final_snapshot = true # in produzione sarebbe da evitare
289     apply_immediately   = true
290
291     tags = { Name = "${var.project}-db" }
292 }
293
294 # --- ElastiCache Redis: tabellone cucina ---
295 resource "aws_elasticache_cluster" "redis" {
296     cluster_id      = "${var.project}-redis"
297     engine          = "redis"
298     node_type       = "cache.t3.micro"
299     num_cache_nodes = 1
300     parameter_group_name = "default.redis7"
301     port            = 6379
302
303     subnet_group_name = aws_elasticache_subnet_group.main.name
304     security_group_ids = [aws_security_group.data.id]
305
306     tags = { Name = "${var.project}-redis" }
307 }
308
309 # --- Amazon MQ per RabbitMQ: coda eventi ordine ---
310 # AWS non supporta piu' mq.t3.micro con RabbitMQ: mq.m7g.medium e' il taglio
311 # piu' economico disponibile. Amazon MQ usa AMQPS (TLS) sulla porta 5671,
non 5672 in chiaro
312 resource "aws_mq_broker" "rabbitmq" {
313     broker_name      = "${var.project}-mq"
314     engine_type      = "RabbitMQ"
315     engine_version    = "3.13"
316     host_instance_type = "mq.m7g.medium"
317     deployment_mode   = "SINGLE_INSTANCE"

```

```

318
319     subnet_ids          = [aws_subnet.public[0].id]
320     security_groups      = [aws_security_group.data.id]
321     publicly_accessible  = false
322     auto_minor_version_upgrade = true
323
324     user {
325         username = "mensa"
326         password = random_password.mq.result
327     }
328
329     tags = { Name = "${var.project}-mq" }
330 }
331
332 # -- SSM Parameter Store: connection string dei servizi gestiti --
333 # Lo script di deploy le legge da qui e crea il Secret Kubernetes:
334 # non viene caricata nessuna credenziale nel repo o nei manifest
335 resource "aws_ssm_parameter" "database_url" {
336     name = "/${var.project}/DATABASE_URL"
337     type = "SecureString"
338     value =
339         "postgresql://mensa:${random_password.db.result}@${aws_db_instance.postgres.address}:5
340
341 resource "aws_ssm_parameter" "redis_url" {
342     name = "/${var.project}/REDIS_URL"
343     type = "String"
344     value =
345         "redis://${aws_elasticache_cluster.redis.cache_nodes[0].address}:6379/0"
346
347 resource "aws_ssm_parameter" "rabbitmq_url" {
348     name = "/${var.project}/RABBITMQ_URL"
349     type = "SecureString"
350     # l'endpoint di Amazon MQ arriva gia' come "amqps://host:5671"
351     value =
352         "amqps://mensa:${random_password.mq.result}@${replace(aws_mq_broker.rabbitmq.instances
353         "amqps://", "")}"
354
355 # ----- computing
356 # Ci sono Tre istanze EC2 (1 control plane + 2 worker) hanno lo stesso ruolo
357 # delle VM su local
358
359 # AMI Ubuntu 22.04 piu' recente
360 data "aws_ami" "ubuntu" {
361     most_recent = true
362     owners      = ["099720109477"] # Canonical
363
364     filter {
365         name = "name"
366         values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
367     }
368 }
369
370 # La chiave SSH del PC viene caricata su AWS e messa nelle istanze
371 resource "aws_key_pair" "main" {
372     key_name = "${var.project}-key"
373     public_key = trimspace(file(pathexpand(var.ssh_public_key_path)))
374 }
375

```

```

374 # Ruolo IAM istanze, fa scaricare immagini da ECR e usare S3 senza mettere
    credenziali nel codice o nei pod
375 resource "aws_iam_role" "node" {
376     name = "${var.project}-node-role"
377
378     assume_role_policy = jsonencode({
379         Version = "2012-10-17"
380         Statement = [{
381             Effect      = "Allow"
382             Principal   = { Service = "ec2.amazonaws.com" }
383             Action      = "sts:AssumeRole"
384         }]
385     })
386 }
387
388 resource "aws_iam_role_policy_attachment" "ecr_read" {
389     role           = aws_iam_role.node.name
390     policy_arn     = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
391 }
392
393 resource "aws_iam_role_policy" "s3_images" {
394     name = "${var.project}-s3-images"
395     role = aws_iam_role.node.id
396
397     policy = jsonencode({
398         Version = "2012-10-17"
399         Statement = [{
400             Effect      = "Allow"
401             Action      = ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"]
402             Resource    = "${aws_s3_bucket.images.arn}/*"
403         }]
404     })
405 }
406
407 resource "aws_iam_instance_profile" "node" {
408     name = "${var.project}-node-profile"
409     role = aws_iam_role.node.name
410 }
411
412 resource "aws_instance" "control_plane" {
413     ami                = data.aws_ami.ubuntu.id
414     instance_type      = var.instance_type
415     subnet_id          = aws_subnet.public[0].id
416     vpc_security_group_ids = [aws_security_group.nodes.id]
417     key_name           = aws_key_pair.main.key_name
418     iam_instance_profile = aws_iam_instance_profile.node.name
419
420     root_block_device {
421         volume_size = 20
422     }
423
424     tags = { Name = "${var.project}-cp" }
425 }
426
427 resource "aws_instance" "worker" {
428     count              = var.worker_count
429     ami               = data.aws_ami.ubuntu.id
430     instance_type      = var.instance_type
431     subnet_id         = aws_subnet.public[count.index %
        length(aws_subnet.public)].id
432     vpc_security_group_ids = [aws_security_group.nodes.id]
433     key_name           = aws_key_pair.main.key_name

```



```

434     iam_instance_profile    = aws_iam_instance_profile.node.name
435
436     root_block_device {
437         volume_size = 20
438     }
439
440     tags = { Name = "${var.project}-worker-${count.index + 1}" }
441 }
442
443 # ----+++ QUI C'E' IL NETWORK LOAD BALANCER: unico punto di ingresso
444 #         pubblico dell'app +++----
445 resource "aws_lb" "frontend" {
446     name                = "${var.project}-nlb"
447     load_balancer_type = "network"
448     subnets            = aws_subnet.public[*].id
449
450     tags = { Name = "${var.project}-nlb" }
451 }
452 # Il target group punta alla NodePort 30080 dei worker
453 resource "aws_lb_target_group" "frontend" {
454     name        = "${var.project}-tg"
455     port        = 30080
456     protocol    = "TCP"
457     vpc_id      = aws_vpc.main.id
458     target_type = "instance"
459
460     health_check {
461         protocol = "TCP"
462     }
463 }
464
465 resource "aws_lb_target_group_attachment" "workers" {
466     count          = var.worker_count
467     target_group_arn = aws_lb_target_group.frontend.arn
468     target_id      = aws_instance.worker[count.index].id
469     port          = 30080
470 }
471
472 resource "aws_lb_listener" "http" {
473     load_balancer_arn = aws_lb.frontend.arn
474     port              = 80
475     protocol          = "TCP"
476
477     default_action {
478         type = "forward"
479         target_group_arn = aws_lb_target_group.frontend.arn
480     }
481 }
482
483 # ---- ECR: registry privato per le immagini dei servizi ----
484 resource "aws_ecr_repository" "services" {
485     for_each = toset(["menu-service", "order-service", "kitchen-service",
486                     "frontend"])
487
488     name                = "${var.project}/${each.key}"
489     image_tag_mutability = "MUTABLE"
490     force_delete        = true # comodo in un progetto didattico: destroy
491                             # senza svuotare a mano
492 }
493
494 # -- Inventory Ansible generato con gli IP reali delle istanze --

```

```

493 resource "local_file" "ansible_inventory" {
494     filename = "${path.module}/ansible/inventory.ini"
495
496     content = <<-EOT
497     [control_plane]
498     ${var.project}-cp ansible_host=${aws_instance.control_plane.public_ip}
499
500     [workers]
501     ${for i, w in aws_instance.worker~}
502     ${var.project}-worker-${i + 1} ansible_host=${w.public_ip}
503     ${endfor~}
504
505     [k8s_cluster:children]
506     control_plane
507     workers
508
509     [k8s_cluster:vars]
510     ansible_user=ubuntu
511     ansible_ssh_private_key_file=${pathexpand(replace(var.ssh_public_key_path,
512         ".pub", ""))}
512     ansible_ssh_common_args='-o StrictHostKeyChecking=no'
513 EOT
514 }
515
516 # ----- output
517
518 output "control_plane_ip" {
519     value = aws_instance.control_plane.public_ip
520 }
521
522 output "worker_ips" {
523     value = aws_instance.worker[*].public_ip
524 }
525
526 output "app_url" {
527     description = "URL pubblico dell'app (Network Load Balancer)"
528     value       = "http://${aws_lb.frontend.dns_name}"
529 }
530
531 output "s3_bucket_images" {
532     value = aws_s3_bucket.images.bucket
533 }
534
535 output "rds_endpoint" {
536     value = aws_db_instance.postgres.address
537 }
538
539 output "redis_endpoint" {
540     value = aws_elasticache_cluster.redis.cache_nodes[0].address
541 }
542
543 output "mq_endpoint" {
544     value = aws_mq_broker.rabbitmq.instances[0].endpoints[0]
545 }
546
547 output "ecr_registry" {
548     description = "Registry ECR per docker login e push"
549     value       =
550         "${data.aws_caller_identity.current.account_id}.dkr.ecr.${var.region}.amazonaws.com"
551 }
552 data "aws_caller_identity" "current" {}

```

```

553
554 # Le foto dei piatti devono essere visibili dal browser: policy di sola
555 # lettura sugli oggetti. Le chiavi sono uuid casuali e nessuno puo' scrivere
556 # nel bucket dall'esterno. In alternativa si sarebbero potuti usare presigned
557 # URL, tenendo il bucket completamente privato.
558 resource "aws_s3_bucket_policy" "images_public_read" {
559     bucket      = aws_s3_bucket.images.id
560     depends_on = [aws_s3_bucket_public_access_block.images]
561
562     policy = jsonencode({
563         Version = "2012-10-17"
564         Statement = [{
565             Sid      = "PublicReadGetObject"
566             Effect    = "Allow"
567             Principal = "*"
568             Action     = "s3:GetObject"
569             Resource  = "${aws_s3_bucket.images.arn}/*"
570         }]
571     })
572 }
573
574 # ----- CI/CD
575 # GitHub Actions si autentica su AWS con OIDC: niente chiavi statiche nei
576 # secret del repository, solo credenziali temporanee disponibili per 1 ora
577
578 variable "github_repo" {
579     description = "owner/repo autorizzato ad assumere il ruolo"
580     type        = string
581     default     = "vitomarin02-del/Cloud-Mensa-AWS-Version"
582 }
583
584 resource "aws_iam_openid_connect_provider" "github" {
585     url                  = "https://token.actions.githubusercontent.com"
586     client_id_list      = ["sts.amazonaws.com"]
587     thumbprint_list     = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
588 }
589
590 # Il ruolo puo' essere assunto SOLO dai workflow di questo repository
591 resource "aws_iam_role" "github_actions" {
592     name = "${var.project}-github-actions"
593
594     assume_role_policy = jsonencode({
595         Version = "2012-10-17"
596         Statement = [{
597             Effect    = "Allow"
598             Principal = { Federated = aws_iam_openid_connect_provider.github.arn }
599             Action     = "sts:AssumeRoleWithWebIdentity"
600             Condition = {
601                 StringEquals = {
602                     "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
603                 }
604                 StringLike = {
605                     "token.actions.githubusercontent.com:sub" = [
606                         "repo:${var.github_repo}:",
607                         "repo:vito02-del@*/Cloud-Mensa-AWS-Version@*:"
608                     ]
609                 }
610             }
611         }]
612     })
613 }

```

```

614 # Permessi minimi: push su ECR e invio comandi al control plane via SSM
615 resource "aws_iam_role_policy" "github_actions" {
616   name = "${var.project}-github-actions-policy"
617   role = aws_iam_role.github_actions.id
618
619   policy = jsonencode({
620     Version = "2012-10-17"
621     Statement = [
622       {
623         Effect = "Allow"
624         Action = ["ecr:GetAuthorizationToken"]
625         Resource = "*"
626       },
627       {
628         Effect = "Allow"
629         Action = [
630           "ecr:BatchCheckLayerAvailability",
631           "ecr:CompleteLayerUpload",
632           "ecr:InitiateLayerUpload",
633           "ecr:PutImage",
634           "ecr:UploadLayerPart",
635           "ecr:BatchGetImage"
636         ]
637         Resource = [for r in aws_ecr_repository.services : r.arn]
638       },
639       {
640         Effect = "Allow"
641         Action = ["ec2:DescribeInstances"]
642         Resource = "*"
643       },
644       {
645         Effect = "Allow"
646         Action = ["ssm:SendCommand", "ssm:GetCommandInvocation",
647           "ssm:ListCommandInvocations"]
648         Resource = "*"
649       }
650     ]
651   })
652 }
653
654 # Serve affinché SSM possa eseguire comandi sulle istanze
655 resource "aws_iam_role_policy_attachment" "ssm_core" {
656   role = aws_iam_role.node.name
657   policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
658 }
659
660 output "github_actions_role_arn" {
661   description = "Da inserire nel secret AWS_ROLE_ARN del repository"
662   value = aws_iam_role.github_actions.arn
663 }

```

Listing A.6: aws/main.tf

A.7 Pipeline CI/CD

```

1 name: build e deploy su AWS
2
3 # Ad ogni push su main: build delle immagini, push su ECR e aggiornamento
4 # dei Deployment sul cluster.
5 on:

```

```

6  push:
7    branches: [main]
8    workflow_dispatch: # avvio manuale dalla scheda Actions
9
10  env:
11    AWS_REGION: eu-central-1
12    PROJECT: mensa
13
14  permissions:
15    id-token: write # necessario per OIDC
16    contents: read
17
18  jobs:
19    deploy:
20      runs-on: ubuntu-latest
21
22      steps:
23        - uses: actions/checkout@v4
24
25        - name: Debug claim OIDC
26          run: |
27            TOKEN=$(curl -sH "Authorization: bearer
28              $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
29              "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sts.amazonaws.com" | jq
30                -r .value)
31            echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq '{sub,
32              aud}'
33
34            # Nessuna chiave AWS: il token OIDC di GitHub viene scambiato con
35            # credenziali temporanee del ruolo IAM.
36        - name: Autenticazione su AWS (OIDC)
37          uses: aws-actions/configure-aws-credentials@v4
38          with:
39            role-to-assume: ${ secrets.AWS_ROLE_ARN }
40            aws-region: ${ env.AWS_REGION }
41
42        - name: Login su ECR
43          id: ecr
44          uses: aws-actions/amazon-ecr-login@v2
45
46        - name: Build e push delle immagini
47          env:
48            REGISTRY: ${ steps.ecr.outputs.registry }
49            TAG: ${ github.sha } # tag = commit, così ogni deploy è
50              tracciabile
51          run: |
52            for s in menu-service order-service kitchen-service frontend; do
53              docker build -t $REGISTRY/$PROJECT/$s:$TAG ./s
54              docker push $REGISTRY/$PROJECT/$s:$TAG
55            done
56
57            # L'API server accetta connessioni solo dall'IP dello sviluppatore,
58            # quindi il deploy non parte dal runner: si esegue kubectl sul control
59            # plane tramite SSM Run Command.
60        - name: Aggiorna i Deployment sul cluster
61          env:
62            REGISTRY: ${ steps.ecr.outputs.registry }
63            TAG: ${ github.sha }
64          run: |
65            CP_ID=$(aws ec2 describe-instances \
66              --filters "Name=tag:Name,Values=$PROJECT-cp"
67              "Name=instance-state-name,Values=running" \

```

```

63     --query "Reservations[0].Instances[0].InstanceId" --output text)
64     echo "control plane: $CP_ID"
65
66     CMD=$(aws ssm send-command \
67     --instance-ids "$CP_ID" \
68     --document-name "AWS-RunShellScript" \
69     --comment "deploy $TAG" \
70     --parameters commands="[
71     \"export KUBECONFIG=/etc/kubernetes/admin.conf\",
72     \"kubectl -n mensa delete secret ecr-creds
73     --ignore-not-found\",
74     \"kubectl -n mensa create secret docker-registry ecr-creds
75     --docker-server=$REGISTRY --docker-username=AWS
76     --docker-password=\"$(aws ecr get-login-password --region
77     $AWS_REGION)\",
78     \"kubectl -n mensa set image deployment/menu-service
79     menu-service=$REGISTRY/$PROJECT/menu-service:$TAG\",
80     \"kubectl -n mensa set image deployment/order-service
81     order-service=$REGISTRY/$PROJECT/order-service:$TAG\",
82     \"kubectl -n mensa set image deployment/kitchen-service
83     kitchen-service=$REGISTRY/$PROJECT/kitchen-service:$TAG\",
84     \"kubectl -n mensa set image deployment/frontend
85     frontend=$REGISTRY/$PROJECT/frontend:$TAG\",
86     \"kubectl -n mensa rollout status deployment/frontend
87     --timeout=180s\"
88     ]" \
89     --query "Command.CommandId" --output text)
90
91     echo "comando SSM: $CMD"
92     sleep 60
93     aws ssm get-command-invocation --command-id "$CMD" --instance-id
94     "$CP_ID" \
95     --query
96     "{stato:Status,output:StandardOutputContent,errore:StandardErrorContent}"

```

Listing A.7: .github/workflows/deploy.yml

A.8 Script di deployment

```

1  #!/bin/bash
2  # Deploy dell'app sul cluster EC2.
3  # Crea i due Secret necessari (credenziali dei servizi gestiti e token ECR)
4  # e applica i manifest. Le password non sono mai nel repo: si leggono da SSM
5  # +++ Uso: ./deploy.sh
6  set -e
7
8  REGION="eu-central-1"
9  REGISTRY=$(terraform output -raw ecr_registry)
10 export KUBECONFIG="$PWD/ansible/kubeconfig"
11
12 kubectl apply -f k8s/00-namespace.yaml
13
14 # --- Secret con le connection string lette da SSM Parameter Store ---
15 DATABASE_URL=$(aws ssm get-parameter --name /mensa/DATABASE_URL
16     --with-decryption --region $REGION --query Parameter.Value --output text)
17 REDIS_URL=$(aws ssm get-parameter --name /mensa/REDIS_URL --region $REGION
18     --query Parameter.Value --output text)
19 RABBITMQ_URL=$(aws ssm get-parameter --name /mensa/RABBITMQ_URL
20     --with-decryption --region $REGION --query Parameter.Value --output text)

```

```
19 kubectl -n mensa create secret generic mensa-secrets \  
20 --from-literal=DATABASE_URL="$DATABASE_URL" \  
21 --from-literal=REDIS_URL="$REDIS_URL" \  
22 --from-literal=RABBITMQ_URL="$RABBITMQ_URL" \  
23 --dry-run=client -o yaml | kubectl apply -f -  
24  
25 # --- Secret per scaricare le immagini da ECR. +++ RICORDA: token valido  
    solo 12 ore  
26 kubectl -n mensa create secret docker-registry ecr-creds \  
27 --docker-server=$REGISTRY \  
28 --docker-username=AWS \  
29 --docker-password="$(aws ecr get-login-password --region $REGION)" \  
30 --dry-run=client -o yaml | kubectl apply -f -  
31  
32 kubectl apply -f k8s/  
33  
34 echo  
35 echo "App in arrivo su: $(terraform output -raw app_url)"  
36 kubectl -n mensa get pods
```

Listing A.8: deploy.sh